SOLID principi dijagrama klasa

S - Single Responsibility Principle

Svaka metoda/klasa mora imati samo jednu odgovornost

Svaka klasa je cjelina za sebe i svaka metoda unutar pojedine klase ima jasno definisanu svrhu. Klase poštuju objektno orijentisanu paradigmu u smislu da se funkcionalnosti vežu za one objekte na koje se stvarno odnose. Ovo je razlog postojanja odvojenih klasa kao što su SmartPlanner, Recenzija, Newsletter, Atrakcija, UgostiteljskiObjekat i Event.

Na primjer, funkcionalnost planiranja obilaska nije smještena u klasu RegistrovaniKorisnik, nego je izdvojena u klasu SmartPlanner, kako bi se planiranje tretiralo kao posebna odgovornost sistema. Isto tako, recenzije su izdvojene u klasu Recenzija, a obavještenja korisnicima u klasu Newsletter, čime se izbjegava da klasa korisnika preuzima odgovornosti koje joj ne pripadaju. Na taj način svaka klasa ostaje fokusirana na svoju osnovnu namjenu, a sistem postaje pregledniji i lakši za održavanje.

O – Open/Closed Principle

Klase moraju biti otvorene za proširenja, a zatvorene za modifikaciju.

Klase sadrže osnovne atribute i veze koje opisuju njihovu glavnu ulogu u sistemu, čime su pogodne za dalja proširenja bez potrebe za čestim izmjenama postojeće strukture. Primjer toga su enumeracije VrstaAtrakcije, VrstaObjekta i VrstaEventa, koje je moguće proširiti dodavanjem novih tipova atrakcija, ugostiteljskih objekata ili događaja.

Na taj način, ukoliko se u sistemu pojavi nova kategorija atrakcije, nova vrsta objekta ili novi tip eventa, nije potrebno mijenjati osnovne klase Atrakcija, UgostiteljskiObjekat i Event. Dovoljno je proširiti postojeće enumeracije odgovarajućim vrijednostima. Ovim se postiže fleksibilnost sistema, jer se nove funkcionalnosti mogu dodavati bez narušavanja već definisanog modela klasa.

L – Liskov Substitution Principle

Izvedene klase se trebaju moći proslijediti svugdje gdje može i bazna klasa.

Postojanje generalizacijske veze između klase Korisnik i izvedenih klasa Admin, Operater i RegistrovaniKorisnik omogućava poštovanje ovog principa. Sve navedene klase predstavljaju

posebne oblike korisnika sistema, pa se mogu koristiti svugdje gdje se očekuje objekat tipa Korisnik.

Primjer toga je funkcionalnost autentifikacije ili identifikacije korisnika u sistemu. Iako Admin, Operater i RegistrovaniKorisnik imaju različite uloge i nivoe pristupa, svi oni i dalje predstavljaju korisnike aplikacije. Zbog toga se operacije koje se odnose na opšti pojam korisnika mogu primijeniti i na njihove izvedene klase, bez narušavanja logike sistema.

I – Interface Segregation Principle

Klijent ne treba ovisiti od interfejsa sa metodama koje nikada neće koristiti.

Funkcionalnosti su razdvojene tako da svaka klasa koristi samo one metode i podatke koji su vezani za njenu ulogu u sistemu. RegistrovaniKorisnik je jedina korisnička klasa koja ima potrebu za funkcionalnostima kao što su kreiranje planova obilaska i pisanje recenzija, pa su te funkcionalnosti povezane upravo s klasama SmartPlanner i Recenzija.

S druge strane, klase Admin i Operater ne moraju biti opterećene funkcionalnostima koje se odnose na korisničke planove ili recenzije. Isto tako, slanje obavještenja korisnicima izdvojeno je u posebnu klasu Newsletter, umjesto da se ta odgovornost dodjeljuje svim korisničkim klasama. Na taj način se izbjegava pretrpavanje klasa metodama koje ne koriste, a sistem ostaje jasniji, modularniji i lakši za održavanje.

D – Dependency Inversion Principle

Umjesto da konkretne klase ovise o konkretnim klasama, trebaju ovisiti o interfejsima ili apstraktnim klasama.

Ovim principom se postiže smanjenje direktne zavisnosti između klasa i olakšava povezivanje funkcionalnosti unutar sistema. U ovom primjeru se princip može prepoznati kroz postojanje opšte klase Korisnik, iz koje se izvode konkretnije klase Admin, Operater i RegistrovaniKorisnik. Na taj način sistem ne mora uvijek raditi direktno sa konkretnim korisničkim ulogama, već se može oslanjati na opšti pojam korisnika.

Također, funkcionalnosti kao što su planiranje obilaska, recenzije i slanje newslettera izdvojene su u posebne klase. Na primjer, Newsletter izlaže funkcionalnost slanja obavještenja korisnicima, bez potrebe da ostale klase znaju detalje načina slanja. Isto tako, SmartPlanner koristi podatke o atrakcijama, ugostiteljskim objektima, eventima i preferencijama, ali njegova glavna

odgovornost ostaje formiranje plana obilaska. Time se klase međusobno manje vežu za konkretne detalje implementacije, što sistem čini fleksibilnijim i lakšim za buduće izmjene.